Rockchip Android Boot Video Developer Guide

ID: RK-KF-YF-E46

Release Version: V1.0.0

Release Date: 2024-09-11

Security Level: □Secret □Internal □Public ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces how to make the boot-up videos.

Product Version

Chip Name	Kernel Version
Support all chipsets	Linux-4.19, Linux-5.10, Linux-6.10

Intended Audience

This document (this guide) is mainly intended for

Technical Support Engineer

Software Development Engineer

Revision History

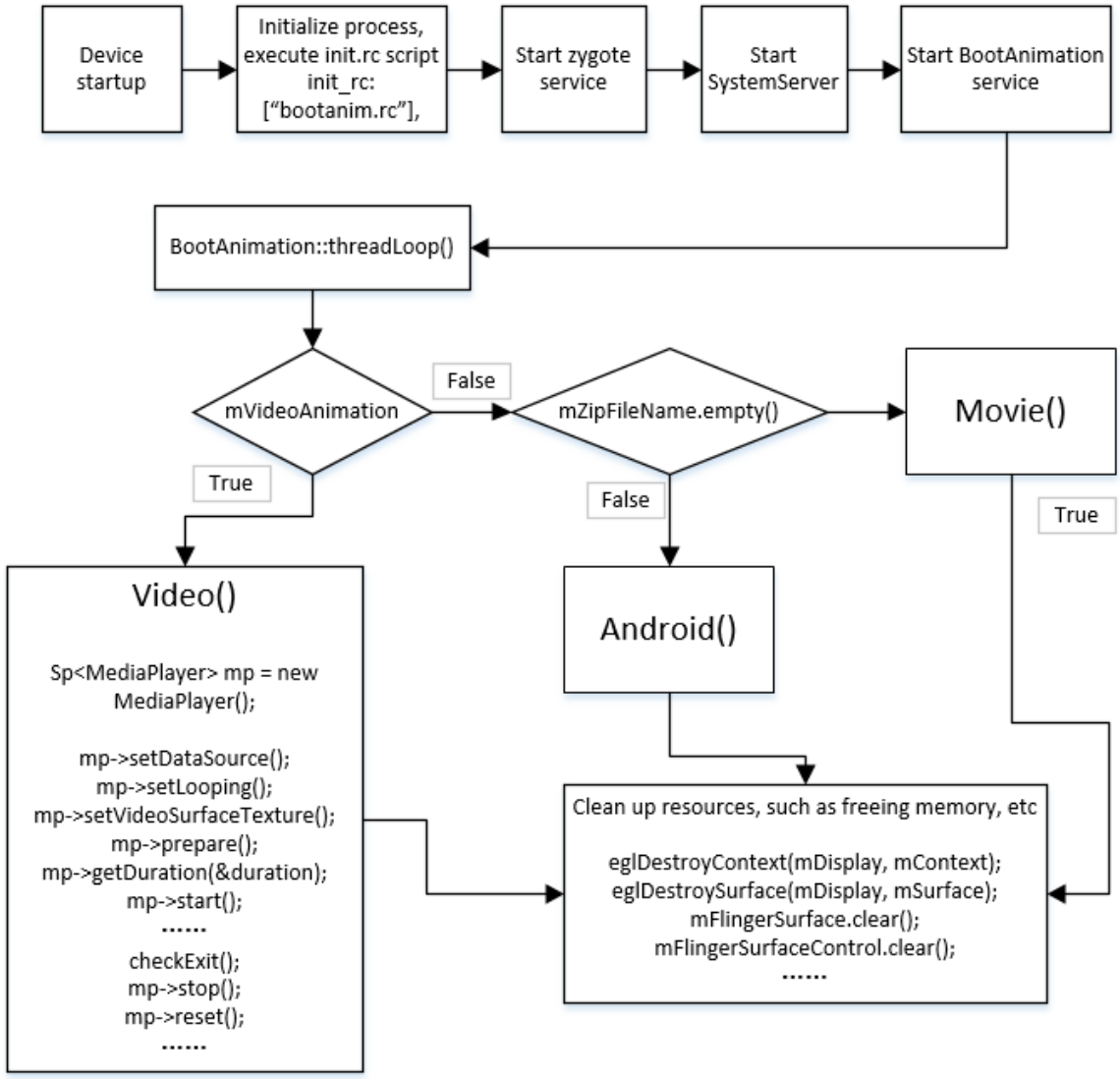
Version	Author	Date	Change Description
V1.0.0	Chen Zulin	2024-9-11	Initial version release

Contents

---

1. Boot Video Principle

Boot animation is part of the Android system startup process, managed by bootanimation. The code path is: frameworks/base/cmds/bootanimation/BootAnimation.cpp. The rk platform adds control over boot video based on the native codebase to implement boot video features. Flowchart is as follows



android() - The system default boot animation

movie() - User-defined boot animation

video() - User-defined boot video

In the findBootAnimationFile function, the boot animation video file bootanimation.ts is retrieved. The video playback is implemented by invoking the Android-provided native MediaPlayer class, and the video function is called during playback. During the animation playback, the checkExit function is called to check whether to exit the animation and return to the Launcher (the main screen interface).

2. Usage

The Android14 SDK currently integrates boot video functionality, which customers can use directly. For detailed instructions, please refer to device/rockchip/common/bootvideo/ReadMe.txt.

2.1 Enable and Disable

Customers can enable the boot video by adding the boot video macro in device/rockchip/common/BoardConfig.mk.

```
Enable Configuration:
BOOT_VIDEO_ENABLE ?= true

Disable Configuration:
BOOT_VIDEO_ENABLE ?= false
```

device/rockchip/common/device.mk

```
ifeq ($(strip $(BOOT_VIDEO_ENABLE)),true)
include device/rockchip/common/bootvideo/bootvideo.mk
endif
```

2.2 Boot Video Storage Location

Place the prepared boot animation video file `bootanimation.ts` into the directory `device/rockchip/common/bootvideo/`.

Note: the default code identifies bootanimation.ts by file name, and other formats such as bootanimation.mp4 can be converted to the ts file type using ffmpeg.

```
ffmpeg -i bootanimation.mp4 bootanimation.ts
```

2.3 Compilation

After full compilation, the bootanimation.ts startup video will be copied to the product/media/ under out directory. For internal code, compiling android individually often fails, therefore, patches are required. It is recommended to follow the operations below during debugging:

1. Push bootanimation.ts to the device's product/media/
2. In the device's product/etc/build.prop, add the following two properties

```
persist.sys.bootvideo.enable=true
persist.sys.bootvideo.showtime=-2
```

Compile the bootanimation module individually and push libbootanimation.so to the device's system/lib64.

2.4 Material Requirements

Videos that can be normally played by the local media center application are eligible as valid materials for boot-up videos. However, since boot-up videos are displayed during the system initialization phase when numerous system services are being launched and the system operates under high-load conditions, it is essential to select videos based on chip capabilities to ensure optimal playback performance. (It is recommended to choose materials with resolutions of 1080p or lower)

2.5 Parameter Control Instructions

Boot video can be controlled via attributes to adjust related parameters:

1. Control boot video feature enable property: persist.sys.bootvideo.enable

```
--true: Enable boot video
--false: Disable boot video
```

2. Set boot video volume property: persist.sys.bootvideo.vol

Parameter range 1-100, indicating volume level, with a maximum volume of 100 scale units.

3. FAQ

3.1 Boot Animation Display Incomplete

After adding bootanimation.zip, there is incomplete display on the right side of the boot video during boot process. Check if frameworks/base/cmds/bootanimation/BootAnimation.cpp contains the following modifications.

```
@@ -563,12 +608,18 @@ status_t BootAnimation::readyToRun() {
     mMaxHeight =
android::base::GetIntProperty("ro.surface_flinger.max_graphics_height", 0);
     ui::Size resolution = displayMode.resolution;
     resolution = limitSurfaceSize(resolution.width, resolution.height);
+    Rect layerStackRect(0, 0, resolution.getWidth(), resolution.getHeight());
+    Rect displayRect(0,0, resolution.getWidth(), resolution.getHeight());
+    SurfaceComposerClient::Transaction t;
+    t.setDisplayProjection(mDisplayToken, ui::ROTATION_0, layerStackRect,
displayRect);
+    t.apply();
     // create the native surface
     sp<SurfaceControl> control = session()-
>createSurface(String8("BootAnimation"),
             resolution.getWidth(), resolution.getHeight(),
PIXEL_FORMAT_RGB_565,
             ISurfaceComposerClient::eOpaque);

-    SurfaceComposerClient::Transaction t;
+    // SurfaceComposerClient::Transaction t;
     if (isValid) {
         // In the case of multi-display, boot animation shows on the specified
displays
         for (const auto id : physicalDisplayIds) {
```

3.2 Boot Animation Stuttering

Some customers may experience stuttering during boot animations, so attempting the boot video solution can be considered. However, when using boot videos, stuttering may still occur due to high system load during bootup. This is also related to the chip's processing capabilities. For instance, the same boot video plays smoothly on rk3588 (due to its powerful CPU performance) but results in stuttering anomalies on rk3568. To address boot video stuttering, it is recommended to implement hardware decoding for the gallery by invoking libhwjpeg. The detailed implementation can refer to the USB camera invocation method.

When using hardware decoding for the gallery, the code implementation should pay attention to adjusting the positions of prepareDecoder (pre-decoding) and flushBuffer (cache release), as follows:
Place them within the constructor and destructor. The prepareDecoder (pre-decoding) only needs to be executed once.

Code modification path: frameworks/base/cmds/bootanimation/BootAnimation.cpp.

```
@@ -214,6 +214,13 @@ BootAnimation::BootAnimation(sp<Callbacks> callbacks)
     } else {
         mShuttingDown = true;
     }
+
+    ret = mHWJpegDecoder.prepareDecoder();
+    if (!ret) {
+        ALOGE("failed to prepare JPEG decoder");
+        mHWJpegDecoder.flushBuffer();
```

```
+         return NO_ERROR;
+     }
      ALOGD("%sAnimationStartTiming start time: %" PRId64 "ms", mShuttingDown ?
"Shutdown" : "Boot",
              elapsedRealtime());
 }
```

```
 BootAnimation::~BootAnimation() {
+     mHWJpegDecoder.flushBuffer();
      if (mAnimation != nullptr) {
          releaseAnimation(mAnimation);
          mAnimation = nullptr;
@@ -405,6 +413,43 @@ status_t BootAnimation::initTexture(FileMap* map, int*
width, int* height,
      return NO_ERROR;
 }
```

Create the initTextureYx function, add an implementation for libhwjpeg, then reference it in playAnimation. The initTexture method is a member function of the BootAnimation class used to initialize the textures employed in the animation.

```
frameworks/base/cmds/bootanimation/BootAnimation.cpp

++status_t BootAnimation::initTextureYx(FileMap* map, int* width, int* height) {
+     int ret = 0;
+     unsigned int input_len = map->getDataLength();
+     char *srcbuf = (char*)map->getDataPtr();
+
+     if (input_len <= 0) {
+         ALOGE("frame size is invalid !");
+         return -1;
+     }
+
+     memset(&mHWDecoderFrameOut, 0, sizeof(MpiJpegDecoder::OutputFrame_t));
+     if ((srcbuf[0] == 0xff) && (srcbuf[1] == 0xd8) && (srcbuf[2] == 0xff)) {
+         ret = mHWJpegDecoder.decodePacket(srcbuf, input_len,
&mHWDecoderFrameOut);
+         if (!ret) {
+             ALOGE("mjpeg decodePacket failed!");
+             mHWJpegDecoder.flushBuffer();
+         }
+     } else {
+         ALOGE("mjpeg data error!!");
+         return -1;
+     }
+     // delete map;
+     const int w = mHWDecoderFrameOut.FrameWidth;
+     const int h = mHWDecoderFrameOut.FrameHeight;
+     uint8_t *p = mHWDecoderFrameOut.MemVirAddr;
+
+     glLoadNv12Img(w, h, p);
+     glDrawNv12Img();
```

```
+
+    *width = w;
+    *height = h;
+
+    mHWJpegDecoder.deinitOutputFrame(&mHWDecoderFrameOut);
+
+    return NO_ERROR;
+}
+
```